

Written Analysis:

Question 1:

Steps:

1. $h(n)$ = mismatches(n) or any position in the word where the current state and goal does not match
2. The lower bounds for this is the minimum number of actions required to get to that goal. This is because the number of mismatched positions between n and goal indicates 1 step needed to change that letter at the mismatched position.
3. It can never be overestimated because every action costs 1 and the remaining cost is the number of mismatches that exist between the n and goal. To change a mismatched position to align with the goal, it will require only one action

Scrabble:

1. $h(n) = \sum \text{Scrabble_Letter_Cost}(\text{goal}[i])$

The formula for scrabble requires summing each index where n and goal mismatches. However, at mismatched position i, the scrabble cost of the letter from goal is summed, not the letter from n.

2. This lower bound is unavoidable because no matter the path it takes to get there from whatever current state, it has to eventually perform the actions that match remaining letters in the goal or else it will never match the goal.
3. It can never be overestimated because the optimal cost path will be at least the sum of the scrabble cost values of each letter in the goal that is mismatched. So it can't overestimate the optimal remaining cost but underestimate since if it makes extra detours to get to the goal that would only add to the total cost which would no longer be optimal.

Frequency:

1. $h(n) = \text{mismatches} * (1 + \min(\text{rarity}(w)))$

The formula for frequency will multiply the number of mismatches between n and goal by the cheapest cost action that exists in the given dictionary.

2. Every time a new word is produced there is some associated cost for that new word. This counts as a single action. Since we can only change one letter at a time it is known then, whatever is the lowest cost word in the dictionary, it can never go lower than that. Multiply that by number of mismatches, since mismatches corresponds with number of actions, and it would

be impossible to go lower than that since the lowest frequency is being multiplied the minimum number of steps required

3. It can never be overestimated because no matter what word is produced it can never be greater than the lowest frequency cost word that exists in the dictionary. Since each new word is equal to a new action taken, that can be multiplied by the lowest frequency cost to get the minimum cost required to get to the goal in those number of steps.

Question 2:

Word Pair	Cost Measure	UCS Cost	UCS Nodes Expanded	A* Cost	A* Nodes Expanded	Reduction
WHY → ASK	Steps	5.00	773	5.00	16	97.93%
WHY → ASK	Scrabble	9.00	765	9.00	13	98.30%
WHY → ASK	Frequency	38.86	808	38.86	754	6.68%
BEAR → DUCK	Steps	4.00	691	4.00	6	99.13%
BEAR → DUCK	Scrabble	11.00	2981	11.00	54	99.80%
BEAR → DUCK	Frequency	38.24	1084	38.24	714	34.13%

1. Yes, UCS and A* always returned the same cost meaning they both found the optimal path
2. Both word pairs had the scrabble cost measure return the largest reduction
3. In both word pairs, frequency was the least informative
4. The heuristic for scrabble is very close to the true cost so it is able to avoid branches that don't lead to the goal. This is seen the clearest in A* search. This also goes for the steps cost measure showing that having the mismatches correspond to the actions left to get to the goal is optimal. The frequency cost measure was the weakest, since having the bound be the lowest cost word in the dictionary results in many possibilities so it has a much looser lower bound. So having it be a larger divergence from the actual cost, it leads to the reduction being smaller.

Question 3:

Steps: Yes, it is always symmetric

Every action taken has a cost of 1 no matter what. So if the cost to go from W_1 to W_2 is x then the cost to go from W_2 to W_1 will also be x because the number of actions required to go between the words is also x no matter the direction

Scrabble: No, not always symmetric

It is not symmetric because the cost of each action is dependent on the letter that was used to replace the letter from the previous state. This means it will not go both ways since reversing the order will lead to different letters being the newly replaced letter

Counterexample: is $\text{dog} \rightarrow \text{cat}$. Going from $\text{dog} \rightarrow \text{cog}$ will have a cost of 3 since c is the new letter.

If you go from $\text{cog} \rightarrow \text{dog}$, it will result in a cost of 2 since d is the new letter.

While the actions are reversible, the symmetry of the cost will be lost

Frequency: No, not always symmetric

Since the cost is dependent on the rarity of the word produced, it changes the cost because the final word produced will be different when it gets reversed.

Counterexample:

$\text{dog} \rightarrow \text{cog} \rightarrow \text{cot} \rightarrow \text{cat}$

vs

$\text{cat} \rightarrow \text{cot} \rightarrow \text{cog} \rightarrow \text{dog}$

While the words cog and cot are the same for both paths the final goal word produced will not be the same which will leave the cost to be different. The actions are still able to be reversed easily.

Question 4:

1.

Model: ChatGPT

Prompt:

For an assignment I have to ask an AI the prompt below:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as

large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

2.

Let $h_0(n)$ = number of positions where n differs from the goal

$r(w)$ = rarity value of word w

g = goal word

$$h(n) = h_0(n) + r(g)$$

3.

Let the current state be n and let $d = h_0(n)$. Since each action changes a single letter that would mean the number of mismatches between n and the goal is equivalent to how many actions are required to reach the goal. So d , the number of actions equals the $h_0(n)$, the number of positions where n differs from the goal.

Each action, d , costs $1 + r(w)$. Which means when the goal word is reached, the final action cost must be $1 + r(g)$.

The 1 in $1 + r(g)$ is already included in the action d .

Using the example from the assignment: $\text{dog} \rightarrow \text{cog} \rightarrow \text{cot} \rightarrow \text{cat}$

$$\text{It would be } (1 + r(\text{cog})) + (1 + r(\text{cot})) + (1 + r(\text{cat})) = 3 + r(\text{cat}).$$

As established previously, the number of mismatches is equal to the number of actions taken, d . So $3 + r(\text{cat})$ is the lower bound cost no matter what. Any path will cost at least this much to get to the cat. Lower bound will require at least 3 actions plus whatever cost the word "cat".

$$3 + r(\text{cat}) \text{ would be } d + r(g) \text{ which is equivalent to } h_0(n) + r(g).$$